

CROSS: Compiler-Driven Optimization of Sparse DNNs Using Sparse/Dense Computation Kernels

Fangxin Liu^{1,2,†}, Shiyuan Huang^{1,2,†}, Ning Yang¹, Zongwu Wang^{1,2}, Haomin Li¹, and Li Jiang^{1,2*}

¹Department of Computer Science and Engineering, Shanghai Jiao Tong University, ²Shanghai Qi Zhi Institute

*Corresponding Author {huangshiyuan, liufangxin, ljiang_cs}@sjtu.edu.cn

Abstract—As deep learning models continue to grow larger and more complex, exploiting sparsity is becoming one of the most critical areas for enhancing efficiency and scalability. Several methods for leveraging sparsity have been proposed to more effectively balance the trade-off between compression ratio and accuracy. While these methods offer algorithmic advantages, they also introduce significant hardware overhead due to index-based encoding and decoding.

In this paper, we propose CROSS, an end-to-end compilation optimization technique to achieve sparse DNN acceleration using GPU computation kernels. The key insight behind CROSS is to exploit parameter distribution locality and reconcile the “sparse” DNN computation with the high-performance “dense” computation kernels. Specifically, we perform an in-depth analysis of sparse operations in mainstream DNN computing frameworks. We then decompose the sparse workload into multiple components to create highly efficient, specialized operators with different sparsity levels. Additionally, we introduce a novel sparse graph translation technique to facilitate computation kernel processing of the sparse workload. The resulting CROSS framework can accommodate various sparsity patterns and optimization techniques, delivering an average $2.03\times$ speedup on inference latency compared to seven state-of-the-art solutions with smaller memory footprints across various models and datasets.

I. INTRODUCTION

With the advent of large-scale models, the number of model parameters has increased significantly, leading to a growing demand for computational and storage resources for model inference. For example, GPT-3 [11] has 175 billion parameters, while Llama-65 [44] has 650 billion. Reducing the size of model parameters without sacrificing accuracy has become a major focus in the industry [27]. Model sparsification, a key approach widely used in both academia and industry, addresses this challenge by reducing the scale of model parameters as deep neural network (DNN) models become large and complex [28].

Model sparsity involves applying specific patterns to tensors in deep neural networks to reduce their size and improve efficiency [25], [29]. One common approach is pruning, which involves setting certain tensor values to zero, achieving row-wise, column-wise, block-wise, or fine-grained element-wise sparsity. With careful pruning, a DNN model can be compressed to a smaller memory footprint without significantly sacrificing accuracy. Custom DNN operators and sparse accelerators are crucial for leveraging sparsity patterns to reduce

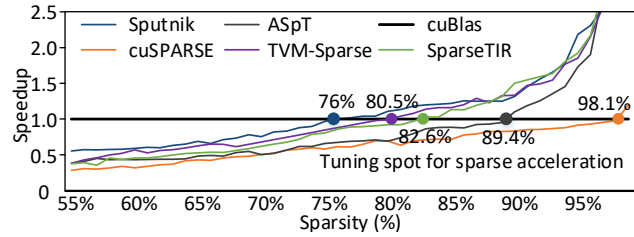


Fig. 1. Speedup of the sparse acceleration scheme compared to cuBlas. Compared with cuBlas, Sputnik, TVM-Sparse, SparseTIR, ASPt, and cuSPARSE can only achieve performance gains when the sparsity rates exceed 76%, 80.5%, 82.6%, 89.4%, and 98.1% respectively. (the sparse matrix is derived from the weight matrix of the sparse bert).

inference latency. For instance, SpAtten [47] proposes token-level pruning in transformers and a dedicated hardware unit for fetching and reordering sparse tokens, while Sanger [31] introduces finer-grained pruning on score matrices and a reconfigurable systolic architecture to support specific sparsity patterns. However, rapidly changing model structures pose a challenge. Designing dedicated sparse accelerators for specific model structures is often impractical and costly, highlighting the need for flexible solutions that adapt to evolving model architectures.

Unfortunately, deep learning systems and compilation frameworks that support sparsity are not yet fully effective at exploiting it. Increased sparsity does not always translate into actual gains in efficiency due to various constraints and limitations.

- First, the computation kernels for general sparse operations are not yet optimal. For example, cuSPARSE [3], the CUDA library for sparse matrix operations, often underperforms compared to cuBLAS [2], its dense counterpart, even when matrix sparsity reaches 98%. To address this, several solutions have been proposed. Sputnik [12] introduces a one-dimensional tiling technique that significantly enhances GPU resource utilization. ASPt [15] improves data locality by reordering column indices. Sparse-Tiled [6] optimizes the tiling method to improve data access efficiency, while SparseTIR [54] uses an innovative composable format for performance tuning. However, experiments show that in scenarios with fine-grained unstructured sparsity, most sparse acceleration schemes fail to outperform dense computation when

[†] These authors contributed equally.

sparsity is below 76% (Figure 1).

- Second, DNN computation typically involves multiple stages, and the sparsity pattern can vary significantly across these stages. This variability makes it challenging to develop sparsity-aware optimizations that yield end-to-end gains. The changing sparsity patterns across different stages necessitate flexible optimization strategies that can adapt to these variations.

In this paper, we propose **CROSS**, a compiler-driven framework for sparse DNN models with fine-grained unstructured sparsity. Our approach addresses two key challenges: 1) First, we observe that there is inherent data locality in the weight matrices of DNN models, resulting in a non-uniform distribution of sparsity. This uneven sparsity means that not all parameters are optimal for representation and computation using a sparse format. To tackle this issue, we introduce a Format-variable Tensor Representation pass, which dynamically selects different storage formats and computational paradigms based on the sparsity characteristics of different matrix regions. 2) Second, we address the imbalance of computational load that arises from using different computational paradigms. We propose two strategies, intra-batch and inter-batch workload balancing strategies. These strategies adjust workload scheduling within individual batches and across multiple batches, ensuring balanced processing and improved efficiency under varying computational paradigms. Specifically, we make the following contributions:

- We analyze the limitations of current sparse optimization schemes in scenarios with low sparsity ratios, identifying that not all parameters are suitable for sparse acceleration and that uneven sparsity distribution causes significant load imbalance among computation cores.
- We introduce the awareness of sparsity to performance modeling, which constructs a Block Execution Time Table (BeTT) to enable the performance prediction of the sparse models with varying sparsity levels.
- We develop the Format-variable Tensor Representation to dynamically adjust block shapes and encoding formats based on the sparsity distribution of the matrix, optimizing both representation and computation efficiency.
- We propose intra-batch and inter-batch workload balancing strategies. Intra-batch balancing assigns dense or sparse computation based on sparsity thresholds, while inter-batch one maps high-load blocks from one batch to low-load blocks in the next to balance the workload.
- We implement these techniques in **CROSS**, an end-to-end compilation framework for accelerating sparse DNNs using sparse & dense GPU computation kernels, achieving up to a $2.03\times$ speedup over the state-of-the-art solution.

II. BACKGROUND

A. Sparsity in DNN

Numerous studies have demonstrated that a substantial amount of sparsity exists in deep neural network (DNN)

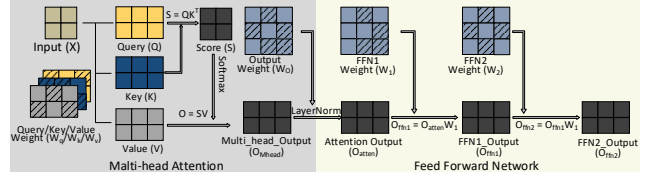


Fig. 2. Sparsity in transformer.

models [26], [50]. Properly applying sparsification can significantly reduce both storage and computational overhead while maintaining model accuracy.

Figure 2 illustrates the sparsity present in current mainstream Transformer-based models. Each encoder/decoder layer in a Transformer consists of two main components: the multi-head attention mechanism and the Feed-Forward network. In the multi-head attention mechanism, sparsity is present in the Query, Key, and Value weights. The Query (Q), Key (K), and Value (V) matrices are computed as follows:

$$Q = X \times W_q, K = X \times W_k, V = X \times W_v, \quad (1)$$

where W_q, W_k , and W_v are the sparse Query/Key/Value weight, respectively. The multi-head attention mechanism then performs operations such as:

$$S = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right), O = S \times V \quad (2)$$

where S is the score matrix, d_k is the dimension size of K , and O represents the output of attention head. These outputs are concatenated to form O_{concat} . Finally, O_{concat} is multiplied by the sparse output weights W_o to produce the final output matrix of the multi-head attention, Q_{atten} .

$$Q_{atten} = O_{concat} \times W_o, \quad (3)$$

After the residual connections and layer normalization, the model proceeds to the linear layers. In the Feed-Forward network, sparsity is present in the weights of both the first linear layer (W_1) and the second linear layer (W_2). The final output matrix of the Feed-Forward network (O_{ffn2}) is obtained through the operations involving these sparse weights.

$$O_{ffn1} = Q_{atten} \times W_1, O_{ffn2} = O_{ffn1} \times W_2, \quad (4)$$

Sparsity is commonly observed in various locations within the Transformer architecture, including the computations described in Equation 1, Equation 3, and Equation 4. This widespread sparsity presents an opportunity to leverage it for accelerating model inference.

B. Sparsity Classification

Among the research works on model sparsification, unstructured sparsity produces fine-grained sparse patterns, which maintains model accuracy but leads to serious degradation of model computational parallelism and memory access efficiency. In particular, the introduction of Tensor Core in

NVIDIA GPUs has significantly enhanced the inference efficiency of structured sparsity compared to unstructured sparsity, leading to increased popularity of structured sparsity in the deep learning community. On the other hand, structured sparsity includes filter-wise [51], channel-wise [13], and block-wise [56] approaches, which involve pruning filters, channels, or sub-blocks of tensors associated with matrix multiplication or activation operations.

However, previous studies [33], [52], [56] have shown that fine-grained structured sparsity, such as block-wise sparsity, struggles to deliver performance gains at low sparsity rates and can cause severe accuracy degradation at high sparsity rates. For example, NVIDIA [52] demonstrates that cuSPARSE [3] Block-SpMM outperforms cuBLAS [2] (dense computation) when the block size exceeds 32 and the sparsity exceeds 60% (with matrix sizes $M=N=K=4096$) on NVIDIA V100 GPUs. However, numerous studies [33], [56] indicate that when the structured sparsity rate exceeds 60%, the model accuracy can decrease by more than 2%.

To address the accuracy degradation caused by structured sparsity at high sparsity rates, researchers have explored unstructured sparsity. cuSPARSE [3] is a widely used library that supports unstructured sparsity formats such as CSR, COO, and CSC. Despite this, studies [58], [59] have shown that cuSPARSE struggles to achieve performance improvements compared to cuBLAS even when sparsity exceeds 96%. Merge-SpMM [53] focuses on latency hiding through instruction-level parallelism and load balancing, achieving significant performance gains over cuSPARSE. ASpT [15] improves performance by reordering matrices and metadata using an adaptive tiling strategy. Sputnik [12] introduces a hierarchical 1-dimensional tiling technique that significantly enhances parallel resource utilization, establishing itself as a state-of-the-art solution for unstructured sparse scenarios. SparseTIR [54] proposes a novel composable format for representing and computing sparse matrices using multiple sparse formats. However, this innovative approach requires manual selection of optimal sparse format combinations, which limits its practical utility.

C. Observations & Motivations

Observation 1: Not all parts of a parameter matrix with unstructured sparsity are suitable for sparse representation and computation. Specifically, Figure 3 illustrates the distribution of sparse rows in Llama-2-7B with a 70% overall sparsity ratio. For instance, in the sparse W_q of Layer 31, 11.4% of rows have densities between 0.38 and 0.40. The distribution is non-uniform, with many rows having densities above 50%, making them unsuitable for sparse representation and computation. Experiments indicate that when local sparsity (assuming uniform distribution) is below 50.1%, the storage overhead of the CSR format exceeds that of dense storage.

Observation 2: Sub-optimal implementation of generic sparse kernels. Generic sparse kernels often use default sparse encoding schemes (e.g., Compressed Sparse Row) for all tensors, which may miss optimization opportunities for

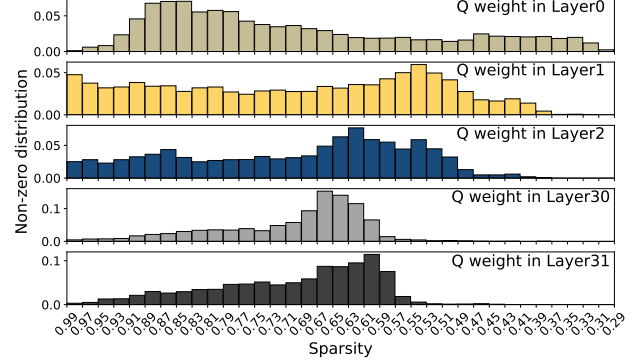


Fig. 3. Non-zero distribution with a 70% overall sparsity ratio in Llama-2-7B. The sparsity of the weight matrix in Llama-2-7B exhibits a severe non-uniform distribution.

specific sparsity patterns. Consequently, fine-grained optimization schemes (e.g., cuSPARSE, Merge-SpMM, ASpT, Sputnik, SparseTIR) underperform compared to cuBLAS when sparsity is below 70% (see Section V-A for details).

Observation 3: Load Imbalance Due to Non-uniform Sparsity Distribution. Taking W_q of layer31 in Figure 3 as an example, rows with over 85% sparsity represent 35.5% of the total, whereas rows with less than 65% sparsity constitute less than 1%. This imbalance results in a higher workload on compute units for sparse computation compared to dense computation. Experiments show stable gains in sparse computation only when sparsity exceeds 85% and minimal gains when sparsity is below 65%, compared to dense computation. Many rows fall between these extremes, exacerbating the imbalance.

Based on **Observations 1 & 2**, selecting appropriate storage formats and computation paradigms according to the local block sparsity can effectively address mismatches between sparsity rates and computation methods. Specifically, blocks with lower sparsity rates should use dense formats for storage and computation, while blocks with higher sparsity rates should use sparse formats. This approach eliminates the shape constraints of the Block Sparse Row (BSR) format, which requires blocks to be square and of uniform size. Furthermore, according to **Observation 3**, we can mitigate load imbalance by choosing suitable storage and computation paradigms for blocks with intermediate sparsity levels. This ensures that load balancing principles are applied, addressing the uneven distribution of computational load.

III. CROSS FRAMEWORK

A. Overview

This section provides an overview of **CROSS**, summarized in Figure 4. The process begins with a pre-trained sparse DNN model as input, which is lowered to the Graph-Level Intermediate Representation (IR). The model parameters can be represented as a dense matrix or in a sparse format, such as CSR.

① In the graph-level IR stage, we perform sparse profiling on the parameter matrix to analyze its sparse distribution.

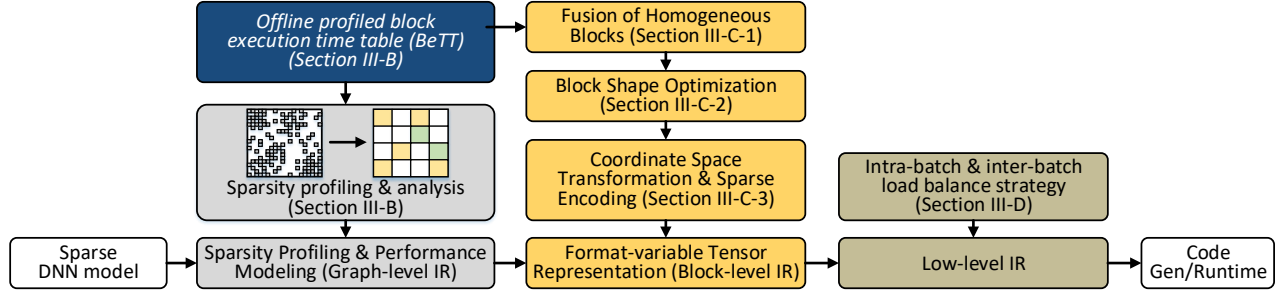


Fig. 4. Overview of CROSS Framework.

This involves block splitting and decomposition into Block-level sub-operators within the SpMM operator. These blocks divide the parameter matrix equally, each having a different sparsity rate. ② An offline profiled Block Execution Time Table (BeTT) is used to guide the fusion of blocks of the same type. BeTT classifies blocks into three types: dense block, swing block, and sparse block. At the block-level IR stage, BeTT attempts to fuse adjacent blocks of the same type into a new block. If the new fused block retains the same type as the original, the fusion is retained; otherwise, it is aborted. ③ Following this, a Block Shape Optimization pass shrinks the block boundary and determines the coordinates of the input feature map corresponding to the parameter block based on its position in the whole matrix. ④ Subsequently, the Coordinate Space Transformation & Sparse Encoding pass transforms the coordinate space of the SpMM operator to that of each post-fusion block, performing sparse format encoding on the sparse and swing blocks. The Block-level IR is then lowered to Low-level IR. ⑤ In the low-level IR, a load balance strategy is implemented to prevent too many blocks from being assigned to the same computing unit, which could lead to load imbalance.

B. Sparsity Profiling & Performance Modeling

This section describes the Sparsity Profiling Pass using the Block Execution Time Table (BeTT), which is designed to optimize the computational efficiency of sparse matrix multiplication by leveraging the non-uniform distribution of nonzero elements. The key insight is to divide the sparse matrix into smaller blocks and determine the optimal computation strategy for each block based on its sparsity ratio. Blocks with higher sparsity are more suited for Sparse Matrix-Matrix Multiplication (SpMM) acceleration, while blocks with lower sparsity benefit from General Matrix-Matrix Multiplication (GEMM) acceleration.

In the sparsity profiling process, a complete sparse matrix is first divided into blocks of equal size according to a predefined granularity, and the sparsity rate of each block is calculated to assess its density of nonzero elements. Then, each block's shape and sparsity rate are used to query the BeTT, which provides the block type and expected execution time based on empirical performance data. This allows for the classification

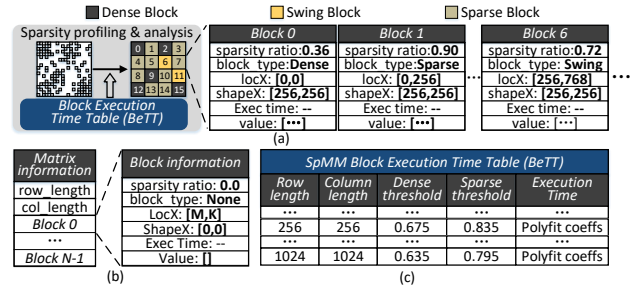


Fig. 5. Sparsity profiling & analysis. (a) the overall sparsity profiling process. (b) sparse matrix information. (c) SpMM Block Execution Time Table (BeTT).

of each block as dense, sparse, or swing, along with its anticipated execution time. Figure 5 (a) illustrates this overall sparsity profiling process.

Figure 5 (b) details the metadata associated with the sparse matrix and its blocks. Each matrix records its row and column dimensions and pointers to individual blocks, while each block includes information such as its sparsity ratio, block type, location within the matrix (*LocX*), shape (*shapeX*), expected execution time, and element values.

To accurately determine block types and execution times, we construct the Block Execution Time Table (BeTT) through extensive offline profiling and data analysis of blocks with various shapes and sparsity rates, as shown in Figure 5 (c). Each entry in the BeTT includes the block's row and column lengths, *Dense threshold*, *Sparse threshold*, and the expected execution time for each configuration. For example, *Block0* in Figure 5 (a) has a shape of [256, 256] and a sparsity rate of 0.36. By querying the BeTT, this block is identified as a dense block, and the expected execution time for this configuration is returned.

The sparsity ratio significantly impacts the execution overhead of SpMM, as depicted in Figure 6. The execution times of both SpMM (implemented by Sputnik) and GEMM (implemented by cuBlas) are plotted against different sparsity ratios. While GEMM execution time remains relatively constant regardless of sparsity, SpMM execution time decreases as sparsity increases. This zonal distribution highlights regions where one computation paradigm outperforms the other.

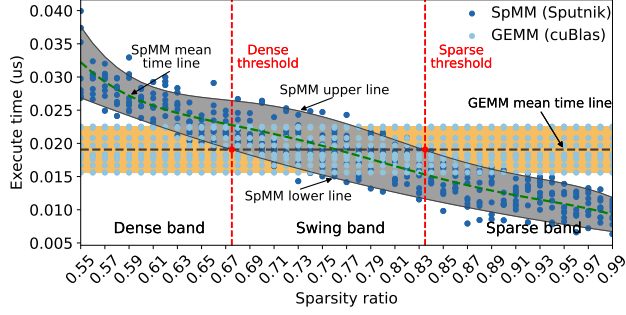


Fig. 6. Runtime distribution of SpMM (implemented by Sputnik) and GEMM (implemented by cuBlas) under different sparsity ratios ($M=K=N=256$, batch=10).

To model execution overheads accurately, we first sample the execution time of SpMM and GEMM under various conditions. We then fit an envelope curve for SpMM's execution time using a polynomial curve fitting algorithm, creating the *SpMM upper line*, *SpMM lower line*, and *SpMM mean time line*, as depicted in Figure 6. The *GEMM mean time line* serves as a baseline for GEMM execution time. The intersection points between these lines define thresholds: the *Dense threshold*, where SpMM offers no gain over GEMM, and the *Sparse threshold*, where SpMM consistently outperforms GEMM. The region between these thresholds is known as the *Swing band*, where either computation paradigm may be viable.

To determine which parts of the sparse matrix are best suited for SpMM or GEMM during static compilation, we use the BeTT to guide this decision. Each block's shape and sparsity rate can be queried to ascertain whether it belongs to the dense, sparse, or swing categories and to predict its execution time. Within the *Swing band* or *Sparse band*, execution time is estimated using the *SpMM mean time line*. In the *Dense band*, execution time aligns with the *GEMM mean time line*. The expected execution time of swing and sparse blocks is calculated using a polynomial curve fitting method for various sparsity rates, as shown in Equation 5.

$$Expected_execution_time = \sum_{i=0}^n a_i x^i, \quad (5)$$

where x is the block's sparsity rate, n is the order of polynomial fitting (set to 5 in our experiments), and $[a_0, \dots, a_{n-1}]$ are the coefficients used to compute the expected execution time based on the sparsity rate. Additionally, an execution time cache stores expected times for commonly used block shapes and sparsity rates, reducing the need for frequent BeTT queries. The resulting BeTT plays a crucial role in subsequent block fusion and workload balancing, ensuring efficient computation across different sparsity conditions.

C. Format-Variable Tensor Representation Pass

After partitioning the sparse matrix into multiple blocks, some block multiplication results need to be accumulated in

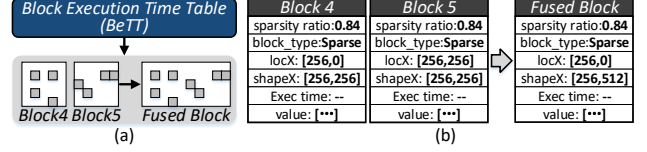


Fig. 7. Block fusion of the same type. (a) The fusion process of blocks with the same type. (b) The block information before and after fusion.

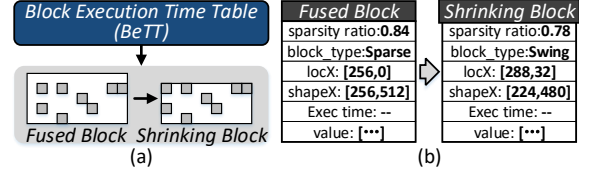


Fig. 8. The shape shrinkage of blocks with boundary sparsity. (a) The block shape shrinking process. (b) The block information before and after block shape shrinking.

the same output coordinate space, leading to data dependencies and forming a block-level graph. To enhance computational efficiency at the block level, we propose Block-level IR optimization.

1) *Fusion of Homogeneous Blocks*: Large blocks are favored in block-level IR due to their improved data locality. However, uniformly shaped blocks might separate adjacent sparse regions with similar sparsity, reducing locality and efficiency. We address this by fusing blocks of the same type. Figure 7 (a) shows this process. For instance, *Block4* and *Block5*, both with a shape of $[256, 256]$ and a sparsity rate of 0.84, are fused if their BeTT queries confirm they are both sparse. If the fusion results in a block that retains its original type, it is kept; otherwise, it is discarded.

2) *Block Shape Optimization*: The boundary regions of blocks, when split and fused, may contain only sparse elements, leading to inefficiencies. To address this, we perform block shape optimization to remove redundant computations. Figure 8 (a) illustrates this process. If a block's boundaries are entirely sparse, it can be compressed, creating a *Shrinking Block*. After shrinking, we update the block's sparsity rate, coordinates, and shape, and query BeTT to determine the block's new type and execution time. The updated block type may influence the chosen computation paradigm.

3) *Coordinate Space Transformation & Sparse Encoding*: Following block fusion and optimization, we map blocks from coordinate space to position space. In graph-level IR, matrix data is indexed by coordinates, while in low-level IR, data is indexed by one-dimensional positions. We transform each two-dimensional block into a one-dimensional format based on its coordinates and shape.

Subsequently, we encode blocks based on their types: dense blocks use the dense format, sparse blocks use the CSR format, and swing blocks use both formats. The *Intra-batch & Inter-batch Load Balancing Strategy* then selects the optimal format for swing blocks according to their computational needs.

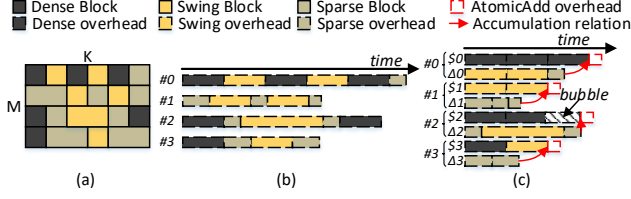


Fig. 9. Intra-batch load balance. (a) A case of a sparse matrix with non-uniform distribution. (b) The execution overhead of imbalanced computational paradigm. (c) Intra-batch load balancing scheduling approach.

D. Intra-batch & inter-batch load balance strategy

After optimizing the block-level operators, the block-level IR is transformed into a lower-level IR for further optimization and acceleration. This section details the strategies for optimizing SpMM at both intra-batch and inter-batch levels.

1) *Intra-batch Load Balancing*: Sparse matrix multiplication can lead to computational imbalances due to the non-uniform distribution of nonzero elements across blocks. Figure 9 (a) illustrates a scenario where a sparse matrix, divided into four block slices, is allocated to four parallel computing units. The computational cost varies significantly among slices: for instance in Figure 9 (b), the first block slice incurs a higher computational cost due to its lower sparsity, while the second and fourth block slices are less costly because of their higher sparsity. Traditional approaches may not efficiently handle these imbalances, particularly when dense and swing blocks are computed using SpMM.

To address this, we propose an intra-batch load balancing strategy. As shown in Figure 9 (c), this strategy involves: 1) SpMM and GEMM Utilization: Compute sparse blocks using SpMM and dense blocks using GEMM; and 2) Swing Block Integration: Integrate swing blocks into the computation paradigm that results in lower costs to balance the load. For blocks within the same slice (e.g., #0 in Figure 9 (c)), results from dense and sparse computations are combined using *AtomicAdd* operations to ensure accuracy. Algorithm 1 provides detailed steps: First, Initialize sets for different block types and their execution latencies (lines 1-5); Then, Categorize blocks and accumulate their expected computation times (lines 6-16); Finally, Assign swing blocks to the set with the shortest expected execution time to achieve balance (lines 17-27).

2) *Inter-batch Load Balancing*: While adjusting swing blocks to dense or sparse computations can mitigate load imbalances within a block slice, imbalances between different block slices in separate batches can still occur. For instance, Figure 9 (c) illustrates that the expected computation time for sparse computation (Δ_0) in block slice 2 (#2) is significantly higher compared to block slice 3 (#3), leading to imbalances across block slices.

To this end, we propose an inter-batch load balancing mechanism. When batch sizes are large, and parallel computing resources cannot fully process all batches simultaneously, batches must be executed serially. If the same computational

Algorithm 1 Intra-batch workload balance strategy

Input: *blocks*; //Blocks within a block slice

Output: *dense_set*, *sparse_set*;

```

1: dense_set =  $\emptyset$ ;
2: swing_set =  $\emptyset$ ;
3: sparse_set =  $\emptyset$ ;
4: dense_lat = 0;
5: sparse_lat = 0;
6: for each block  $\in$  blocks do
7:   if block.type == "Dense" then
8:     dense_set.push_back(block);
9:     dense_lat += block.overhead;
10:  else if block.type == "Sparse" then
11:    sparse_set.push_back(block);
12:    sparse_lat += block.overhead;
13:  else
14:    swing_set.push_back(block);
15:  end if
16: end for
17: for each block  $\in$  swing_set do
18:   if dense_lat  $\geq$  sparse_lat then
19:     sparse_set.insert(block)
20:     sparse_lat += block.overhead;
21:   else
22:     dense_set.insert(block)
23:     dense_lat += block.overhead;
24:   end if
25: end for
26: dense_set.latency = dense_lat;
27: sparse_set.latency = sparse_lat;

```

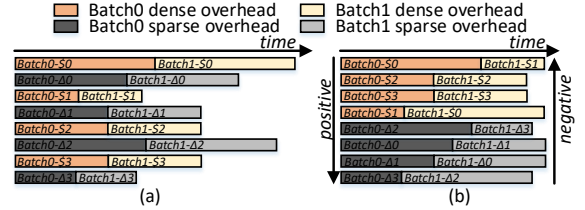


Fig. 10. Inter-batch load balance. (a) The cumulative effect of load imbalance with two batches. (b) Effect of inter-batch load balancing strategy

units handle block slices in different batches, it can exacerbate load imbalances.

Figure 10 (a) shows the cumulative effect of load imbalance with a batch size of 2. For example, block slice 0 in both batches experiences higher computational overhead compared to other slices, leading to inefficient resource utilization. To alleviate this imbalance, we introduce a block slice interleaving strategy. The key idea is to reorder and interleave block slices from different batches to distribute the load more evenly. Figure 10 (b) demonstrates this approach. Implementation of the inter-batch load balancing strategy is detailed in Algorithm 2: 1) Sorting: Sort the execution times of dense and sparse slices for each batch separately. The first batch is

Algorithm 2 Inter-batch workload balance strategy**Input:** *batches*;**Output:** *Concate_batches*;

```

1: direction = Positive;
2: for each batch  $\in$  batches do
3:   sort_slice_by_latency(batch.dense_slices, direction);
4:   sort_slice_by_latency(batch.sparse_slices, direction);
5:   // reverse the sort direction
6:   direction = Reverse(direction);
7: end for
8: batch_len = batches.size(); //the length of batches
9: //the length of slices in one batch
10: slice_len = batches[0].dense_slice_size();
11: k = 0;
12: for i  $\in$  [0, batch_len] do
13:   for j  $\in$  [0, slices_len] do
14:     slice0 = batches[i].dense_slices[j];
15:     slice1 = batches[i + 1].dense_slices[j];
16:     dense_slice = Concatenate(slice0, slice1);
17:     Concate_batches[k].dense_slices[j] = dense_slice;
18:   end for
19:   for j  $\in$  [0, slices_len] do
20:     slice0 = batches[i].sparse_slices[j];
21:     slice1 = batches[i + 1].sparse_slices[j];
22:     sparse_slice = Concatenate(slice0, slice1);
23:     Concate_batches[k].sparse_slices[j] = sparse_slice;
24:   end for
25:   i += 1;
26:   k += 1;
27: end for

```

sorted in ascending order, and subsequent batches are sorted in descending order alternately to ensure balanced distribution (lines 1-8). 2) Concatenation: Concatenate the sorted dense and sparse slices from adjacent batches to form a new batch configuration. This helps to balance the load across different slices by combining the execution times (lines 9-12). 3) Final Assembly: Combine all batches according to the interleaved configuration. If the number of batches is odd, the last batch will not undergo interleaving but will be included as is (lines 13-16).

By applying this inter-batch load balancing strategy, we can reduce the cumulative load imbalance and improve the overall efficiency of batch processing.

IV. EXPERIMENTAL METHODOLOGY

Experiment Setting: We evaluate the performance benefits of **CROSS** on five popular DNN models: BERT-base [10], BERT-large [10], GPT-2 [22], Llama-2 [45], and BART-large [23]. For each model, we consider three different sparsity ratios: 70%, 80%, and 90%. Table I presents the sparsity ratios alongside their corresponding model accuracy.

Each model is trained on a specific dataset: BERT-base and BERT-large are trained on the QQP dataset [8], GPT-2 on DART [36], Llama-2 on LAMBADA [37], and BART-

TABLE I
EVALUATED MODELS WITH DIFFERENT SPARSITY RATIOS AND THEIR
RESULTING ACCURACY (ACC.).

Model	Dataset	sparsity ratio (Sp.)	Acc.(%)
Bert-base [10]	QQP [8]	baseline (0%)	91.20%
		70%	90.85%
		80%	91.15%
		90%	88.13%
Bert-large [10]	QQP	baseline (0%)	91.59%
		70%	91.13%
		80%	90.92%
		90%	89.02%
Bart-large [23]	GLUE/MRPC [48]	baseline (0%)	87.75%
		70%	87.43%
		80%	87.06%
		90%	86.24%
Model	Dataset	sparsity ratio (Sp.)	PPL
GPT-2 [22]	DART [36]	baseline (0%)	15.17
		70%	31.88
		80%	90.40
		90%	188.06
Llama-2-7B [45]	LAMBADA [37]	baseline (0%)	61.87
		68.98%	72.56
		79.18%	74.66
		89.32%	91.24

large on GLUE/MRPC [48]. The evaluation metrics differ depending on the model: accuracy is used for Beert-base, Bert-large, and Bart-large, while perplexity (PPL) is used for GPT-2 and Llama-2. For Bert-base, Bert-large, and Bart-large, the accuracy loss is less than 1% at 70% and 80% sparsity but slightly exceeds 1% at 90% sparsity. For LLM like Llama-2, the PPL increases gradually as the sparsity rate rises.

Baseline: We compare our approach with several existing sparse kernel optimization schemes across various compilation frameworks: 1) cuBlas [2]: A dense linear algebra subroutine library that accelerates AI and HPC applications on GPUs; 2) cuSPARSE [3]: A sparse arithmetic library that supports multiple sparse computing paradigms; 3) ASPT [15]: An innovative optimization scheme for sparse acceleration through matrix and metadata reordering; 4) Sputnik [12]: The state-of-the-art fine-grained unstructured sparse optimization scheme. TVM-Sparse [1]: A sparse optimization solution developed within the TVM framework; 5) SparseTIR [54]: A promising approach to sparse abstraction that requires manual adjustment of sparse format combinations in the coefficient matrix. We specifically use SparseTIR-BSR as the configuration for SparseTIR, as it is recommended for sparse attention and sparse weights.

Implementation: We have seamlessly integrated **CROSS** into the TVM framework, providing a comprehensive solution for sparse inference through multi-level optimizations. Here's how we implemented these optimizations:

- **Graph-Level Optimization:** We have incorporated optimizations into TVM's graph-level IR, allowing for efficient handling of sparsity within the computational graph. This includes strategies for optimizing data flow and computations, ensuring that sparsity patterns are effectively leveraged throughout the model execution.
- **Block-Level Optimization:** At the block level, **CROSS**

employs specific strategies to handle sparse and dense blocks differently. We utilize profiling techniques to identify optimal block sizes and configurations for different sparsity patterns, allowing **CROSS** to dynamically adjust its strategy based on the workload.

- **Low-Level Optimization:** For low-level IR, we incorporate specialized kernel optimizations, where specific instructions and operations are fine-tuned to take full advantage of the underlying hardware capabilities.

During the backend code generation phase, **CROSS** utilizes Sputnik as the foundational sparse block optimization solution. Sputnik provides a baseline for efficiently processing sparse matrices, but **CROSS** is designed to be flexible, allowing for the incorporation of other advanced sparse kernel optimizations as backend solutions. For dense block backend generation, we employ Ansor [57] for performance tuning. Ansor’s auto-scheduling capabilities enable the generation of highly optimized code for dense matrix operations, ensuring that these computations are executed as efficiently as possible. When an operator is too sparse to fully utilize available computation cores, **CROSS** intelligently adapts by merging sparse blocks into a single matrix. This matrix is then executed using a strategy similar to Sputnik, as detailed in Section V-F.

This implementation of **CROSS** within the TVM framework exemplifies a full-stack approach to optimizing sparse matrix computations, leveraging cutting-edge compiler techniques and hardware capabilities to achieve a significant performance improvement.

V. EXPERIMENTAL RESULTS

A. Overall Performance

Inference performance at 90% sparsity: The first line of Figure 11 shows the speedup of five DNN models at 90% sparsity. Under unstructured sparsity, most sparse optimization schemes except cuSPARSE outperform dense schemes like cuBlas. Among them, Sputnik delivers the best performance with speedups of 1.74 $\times$, 2.03 $\times$, 1.44 $\times$, 2.30 $\times$, and 1.94 $\times$ in Bert-base, Bert-large, GPT-2, Llama-2-7B, and Bart-large, separately. The advantage of Sputnik stems from its optimization of both sparse computation and data access using the CSR storage format, enabling positive gains even at lower sparsity levels. In contrast, ASpT, TVM-Sparse, and SparseTIR show slightly lower performance than Sputnik. ASpT has extremely high storage requirements during execution, limiting its performance in large DNN models. TVM-Sparse and SparseTIR also use the CSR format but are not optimized for it, resulting in reduced efficiency. Specifically, SparseTIR faces challenges in balancing model accuracy and performance in fine-grained unstructured sparse scenarios. Notably, cuSPARSE shows the poorest performance, as numerous studies have indicated that it hardly outperforms dense computation, even at high sparsity rates of 96% to 98%.

CROSS leverages the sparse distribution properties in sparse matrices to choose optimal computing paradigms for both dense and sparse blocks. It also selects the most effective

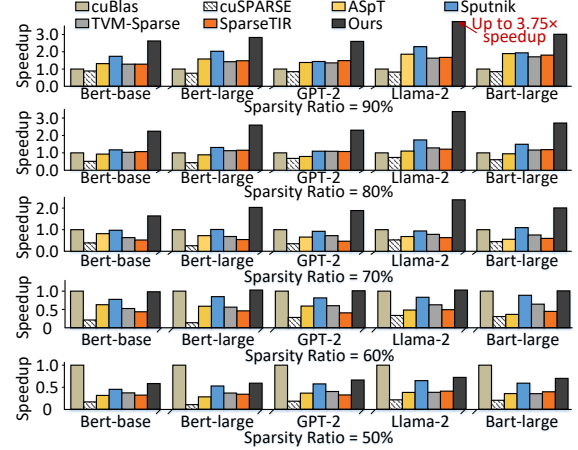


Fig. 11. Inference performance under five different sparsity ratios on NVIDIA A100 GPU (batch size = 32). Compared to cuBlas, we can achieve up to 3.75 $\times$ speedup, and we gain benefits when the sparsity exceeds 60%, while other works require the sparsity to be close to or above 80%.

computing paradigms for blocks in the *Swing band* based on load balancing principles. To be specific, **CROSS** achieves the best gains in Llama-2-7B. This model’s significant non-uniform sparsity distribution provides more opportunities for load balance scheduling. Additionally, Llama-2-7B’s larger hidden size (4096) allows for the use of larger computation blocks, reducing block scheduling overhead. **CROSS** also demonstrates substantial performance gains in Bert-large and Bart-large. The lower gains observed in Bert-large and GPT-2 can be attributed to their smaller hidden sizes (768), which offer fewer opportunities to exploit sparse distribution properties. In GPT-2, the uniformity of the sparse distribution and shared weight matrices in the Query/Key/Value layers further limit the potential speedup from sparsity. Compared to the state-of-the-art Sputnik, **CROSS** achieves speedups of 1.51 $\times$, 1.39 $\times$, 1.81 $\times$, 1.63 $\times$, and 1.56 $\times$ over Sputnik in Bert-base, Bert-large, GPT-2, Llama-2-7B, and Bart-large, respectively.

Inference performance at 80% sparsity: The second line of Figure 11 illustrates the speedup of five DNN models at 80% sparsity. Most sparse compilation optimizations, except for cuSPARSE, achieve positive gains compared to dense computation. Compared to the state-of-the-art Sputnik, **CROSS** achieves even greater speedups, with performance gains of 1.91 $\times$, 1.98 $\times$, 2.10 $\times$, 1.94 $\times$, and 1.82 $\times$ for Bert-base, Bert-large, GPT-2, Llama-2-7B, and Bart-large, respectively.

Inference performance at 70% sparsity: Figure 11 shows the speedup of five DNN models over cuBlas at 70% sparsity. Under unstructured sparsity, most sparse optimization schemes, including cuSPARSE, ASpT, TVM-Sparse, and SparseTIR, fail to outperform dense schemes like cuBlas. Among them, Sputnik delivers the best performance with speedups of 0.97 $\times$, 1.01 $\times$, 0.91 $\times$, 0.95 $\times$, and 1.08 $\times$ for Bert-base, Bert-large, GPT-2, Llama-2-7B, and Bart-large, respectively. Compared to the state-of-the-art Sputnik, **CROSS** achieves speedups of 1.68 $\times$, 2.01 $\times$, 2.08 $\times$, 2.51 $\times$, and 1.84 $\times$

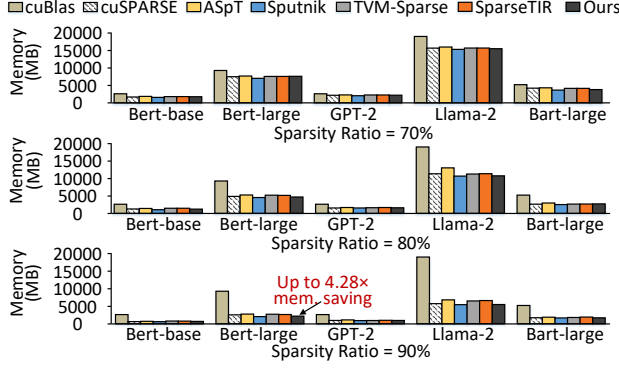


Fig. 12. GPU memory usage under different sparsity levels on NVIDIA A100. Compared to cuBLAS, **CROSS** achieves up to a 4.28 $\times$ memory saving and only adds an average of 6.07% additional storage overhead while delivering higher performance benefits.

in Bert-base, Bert-large, GPT-2, Llama-2-7B, and Bart-large, respectively.

Inference performance at 60% and 50% sparsity: The fourth and fifth lines of Figure 11 show the speedup of five DNN models over cuBLAS at 60% and 50% sparsity, respectively. At 60% sparsity, **CROSS** starts to obtain benefits compared to cuBLAS, while other designs cannot gain benefits. When the sparsity level decreases to 50%, none of the designs achieve any performance benefits. These results suggest that sparsity acceleration requires a sparsity rate greater than 50%. Notably, this level of sparsity can be achieved by almost all DNNs with negligible impact on accuracy [9], [55].

Memory footprints: Figure 12 shows the GPU memory usage across three sparsity ratios on the NVIDIA A100. The six sparse optimization schemes exhibit generally similar memory overheads. ASpT incurs the highest overhead, particularly for Llama-2-7B, due to its reordering transformation. Both cuSPARSE and Sputnik require additional space for runtime caching, though the extra overhead is relatively low. Among these, Sputnik has the least storage overhead.

Compared to Sputnik, **CROSS** incurs 6.5%, 6.0%, and 5.7% additional storage overhead at 70%, 80%, and 90% sparsity, respectively. This is due to the intra-batch load balancing strategy of **CROSS**, which assigns swing blocks to either the dense or sparse set based on load size. This approach may result in some high-sparsity swing blocks being executed in dense block sets. In cases with uniform sparsity distributions (where sparsity in each swing block is evenly distributed), the CSR storage format generally results in lower storage overhead compared to the dense storage format when sparsity exceeds 50.1%. However, if swing blocks are reallocated to dense blocks, additional storage overhead occurs. Despite this, the added storage cost is justified by the performance gains achieved. **CROSS** averages only add 6.07% additional storage overhead on average while delivering higher performance benefits.

Inference performance of different batch sizes: Figure 13 illustrates the inference speedup of Llama-2-7B under

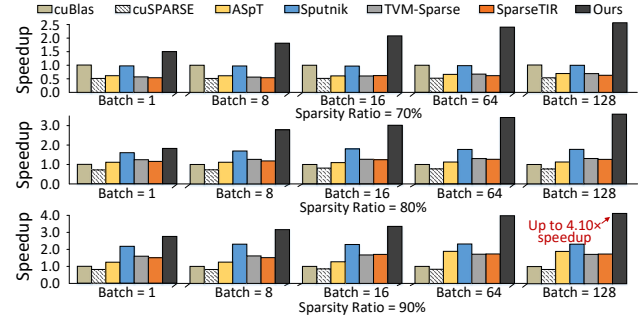


Fig. 13. Inference performance of Llama-2-7B with varying batch sizes on NVIDIA A100. Compared to cuBLAS, **CROSS** achieves up to 4.10 $\times$ speedup.

various batch sizes on the NVIDIA A100 GPU. As batch size increases, sparse optimization schemes other than **CROSS** generally show consistent performance improvements. In contrast, **CROSS** benefits significantly from larger batch sizes due to its inter-batch load balancing strategy. This strategy alleviates load imbalances between batches, enhancing performance with increasing batch sizes. Specifically, with batch sizes ranging from 8 to 64, **CROSS** achieves average speedups of 1.49 $\times$ to 4.10 $\times$ over cuBLAS, 2.57 $\times$ to 4.94 $\times$ over cuSPARSE, 1.65 $\times$ to 3.69 $\times$ over ASpT, 1.14 $\times$ to 2.57 $\times$ over Sputnik, 1.47 $\times$ to 3.72 $\times$ over TVM-Sparse, and 1.58 $\times$ to 4.07 $\times$ over SparseTIR. This demonstrates that **CROSS** scales effectively with larger batch sizes, providing superior performance compared to other sparse optimization schemes.

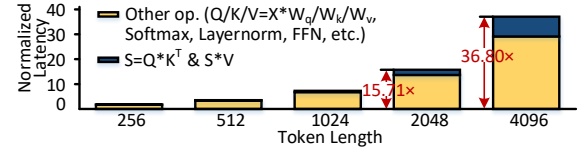


Fig. 14. Normalized inference latency in Llama-2-7B for different token lengths normalized to a token length of 128. These results are obtained with a hidden size of 4096, 70% sparsity, and a batch size of 32.

Figure 14 illustrates the average growth rate of inference latency in the Llama-2-7B model for different token lengths, compared to the latency with a token length of 128. Under the same weight matrix shape and sparse distribution, the computational workload and overhead increase with increasing token length. For example, when the token length is increased from 128 to 2048 (16 $\times$) and 4096 (32 $\times$), the inference latency increases by 15.71 $\times$ and 36.80 $\times$, respectively. This indicates that the latency increases slightly faster than the token length itself. The reason is that in each decode layer, the GEMM operations of $S = Q \times K^T$ and $S \times V$ have two dimensions affected by the token length, while other operators have only one dimension affected. As a result, the computational overhead of these key GEMM operations increases faster as the token length grows.

B. Comparison with previous works on traditional DNNs

Figure 15 illustrates the inference speedup of Vgg-16 [42] (on CIFAR-10 [21]), Resnet-18, and Resnet-50 [14] (on Im-

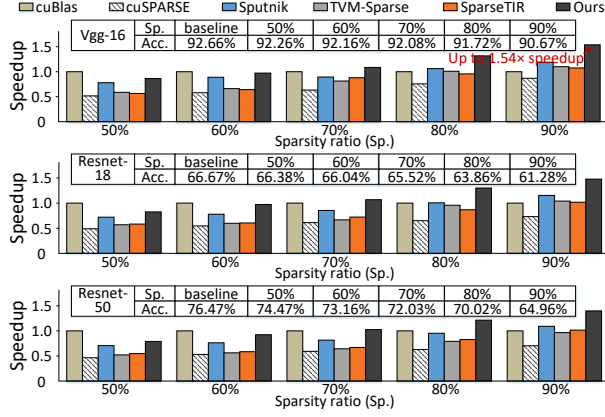


Fig. 15. Inference performance of Vgg-16, Resnet-18, and Resnet-50 with different sparsity ratios on NVIDIA A100 GPU. Compared to cuBlas, we can achieve up to $1.54\times$ speedup, and we gain benefits when the sparsity rate exceeds 70%, while other works need to be close to or above 80%.

ageNet [40]) with different sparsity ratios on the NVIDIA A100 GPU. Currently, various compilation frameworks (e.g., Sputnik and TVM) and vendor libraries (e.g. cuBLAS and cuSPARSE) do not directly optimize convolution operations. Instead, they convert convolutions into matrix multiplications using img2col or Winograd before execution. For a fair comparison, the convolution operators in all CNN models evaluated in this work were split into a combination of img2col + GEMM/SpMM operators. As shown in Figure 15, when the sparsity rate is lower than 60%, none of the approaches can obtain gains compared to cuBLAS. However, when the sparsity rate exceeds 70%, **CROSS** begins to generate positive performance improvements. Sputnik starts to see gains at 80% sparsity, while the remaining methods require sparsity to be close to or exceed 90% before generating benefits. Specifically, at 70%, 80%, and 90% sparsity levels, **CROSS** achieves $1.09\times$, $1.32\times$, and $1.53\times$ performance improvements over cuBLAS in VGG-16. For ResNet-18, the improvements are $1.07\times$, $1.30\times$, and $1.47\times$, and for ResNet-50, the improvements are $1.03\times$, $1.21\times$, and $1.39\times$.

C. Performance of SpMM in different layers.

To further demonstrate the SpMM performance in various layers of the DNN model, we analyze $Q = X \times W_q$, $K = X \times W_k$, and $V = X \times W_v$ for each layer of Llama-2-7B. Figure 16 illustrates the speedup of these SpMM operations in different layers, compared to Sputnik.

In the initial layers of Llama-2-7B, the sparse property is more pronounced due to a non-uniform distribution, resulting in higher speedup ratios for **CROSS**. This effect is particularly notable for the W_q matrix compared to the W_k and W_v matrices, with W_k following in terms of speedup. As the network layers progress, the sparse distribution gradually shifts towards a more uniform distribution, which reduces the acceleration potential of **CROSS**, leading to a decrease in speedup. Overall, the average performance speedups achieved by **CROSS** are $2.90\times$ for $X \times W_q$, $2.42\times$ for $X \times W_k$, and $2.07\times$ for $X \times W_v$.

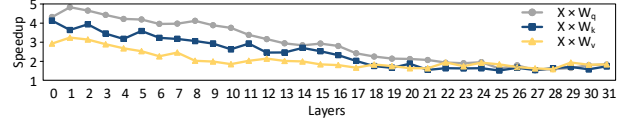


Fig. 16. The speedup of $X \times W_q$, $X \times W_k$, and $X \times W_v$ (SpMM) in each layer of Llama-2-7B.

D. Comparison with Compilation Using Combined Sparse Formats

SparseTIR introduces a composable sparse abstraction utilizing multiple formats for sparse matrix multiplication. We compare **CROSS** against SparseTIR using CSR alone and SparseTIR employing a combination of CSR and BSR with cuBLAS. Figure 17 shows performance improvements for $Q = X \times W_q$, $K = X \times W_k$, and $V = X \times W_v$ in Llama-2-7B. At 70% sparsity, SparseTIR-CSR struggles to outperform cuBLAS, as this sparsity level is below the threshold for significant gains. However, at 80% and 90% sparsity, slight improvements are observed, it shows slight improvements. Using both CSR and BSR formats significantly enhances performance, especially since the BSR format is effective for low-sparsity blocks. **CROSS** further improves sparse computation efficiency by leveraging Sputnik and merging adjacent dense blocks, enhancing data locality and computation efficiency. This results in $1.78 \sim 2.37\times$ performance gains.

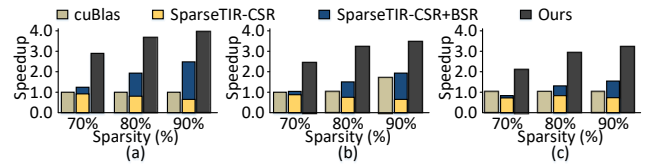


Fig. 17. The speedup of SparseTIR-CSR, SparseTIR-CSR+BSR, and **CROSS** compared to cuBlas. (a) The speedup of SpMM which comes from $Q = X \times W_q$ in Llama-2-7B. (b) The speedup of SpMM which comes from $K = X \times W_k$ in Llama-2-7B. (c) The speedup of SpMM which comes from $V = X \times W_v$ in Llama-2-7B.

E. Ablation Study

We conduct the ablation study to evaluate the effects of the intra-batch and inter-batch load balancing strategies. Figure 18 presents the results for five DNN models at different sparsity levels, with inference performance normalized against cuBlas.

At an average sparsity of 70%, most blocks fall into the *Swing band*, with sparsity ranging from 65% to 80%. This indicates that a significant number of blocks are unsuitable for sparse computation. The intra-batch load balancing strategy optimizes computational paradigms for these swing blocks, yielding performance speedups of $1.35\times$, $1.61\times$, $1.48\times$, $1.95\times$, and $1.71\times$ for Bert-base, Bert-large, GPT-2, Llama-2, and Bart-large, respectively. Due to the load imbalance after intra-batch optimization, the inter-batch load balancing strategy becomes crucial. It achieves performance speedups of $1.64\times$, $2.03\times$, $1.89\times$, $2.38\times$, and $2.01\times$ for the same models.

At 80% sparsity, more blocks are assigned to the *Sparse band*, providing more opportunities for performance gains

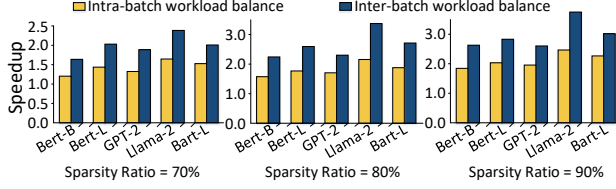


Fig. 18. Ablation study of intra-batch load balance strategy and inter-batch load balance strategy (batch size = 32).

from sparse computations. The intra-batch load balancing strategy now associates a smaller number of blocks as dense but increases the number of swing blocks associated as dense. After intra-batch optimization, **CROSS** achieves performance speedups of $1.66\times$, $1.88\times$, $1.81\times$, $2.31\times$, and $2.01\times$. However, as the sparsity ratio increases, more blocks are associated with sparse blocks, worsening the load imbalance issue. The inter-batch load balancing strategy further mitigates this imbalance, resulting in speedups of up to $3.37\times$.

At 90% sparsity, fewer blocks are associated with dense blocks, making the intra-batch load balancing strategy less effective compared to the inter-batch approach. Nonetheless, due to the increased sparsity, many blocks are either pruned or reduced to very small sizes, which enhances the performance of intra-batch load balancing. After this optimization, **CROSS** achieves performance speedups of $1.88\times$, $2.20\times$, $2.11\times$, $2.64\times$, and $2.56\times$. The inter-batch load balancing strategy, addressing load imbalance issues more effectively, results in speedups of $2.62\times$, $2.83\times$, $2.60\times$, $3.75\times$, and $3.02\times$.

F. Impact on sparsity distribution.

As discussed in Section II-C, sparsity in attention-based DNN models often follows non-uniform distribution patterns, which provides **CROSS** with opportunities to adapt its computational paradigms based on different sparsity characteristics. Figure 19 illustrates how the runtime of $Q = X \times W_q$, $K = X \times W_k$, and $V = X \times W_v$ varies with the sparsity rate in each encoder/decoder layer of Llama-2-7B.

Using cuBlas as a baseline, Sputnik's runtime decreases almost linearly with increasing sparsity when the sparsity is below 95%. The performance improvement becomes more pronounced when sparsity exceeds 95%. In the SpMM operations of Llama-2-7B, Sputnik shows performance gains when sparsity surpasses 77.4% compared to cuBlas (indicated by the dark gray points in Figure 19).

In contrast, **CROSS** shows a more complex relationship with increasing sparsity. As shown in Figure 19 (a), the runtime of $Q = X \times W_q$ starts to decrease rapidly when sparsity is between 55% and 70%. During this range, many blocks that would otherwise be handled inefficiently by sparse computing paradigms are instead processed efficiently by dense computing paradigms, resulting in significant performance gains. The greatest acceleration for **CROSS** is achieved when the sparsity rates of W_q , W_k , and W_v are 68.9%, 72.3%, and 72.4% (*Optimal benefit* in Figure 19), respectively.

As sparsity continues to increase, the runtime decreases more slowly because fewer blocks are suitable for the dense

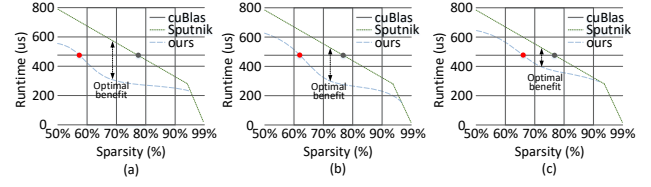


Fig. 19. Average runtime of SpMM in all Llama-2-7B network layers. ($M=K=N=4096$, batch size=32). (a) Average runtime for $Q = X \times W_q$. (b) Average runtime for $K = X \times W_k$. (c) Average runtime for $V = X \times W_v$.

computing paradigm. Once the sparsity exceeds 95.2%, almost no blocks remain suitable for dense computation, and all sparse blocks are combined into a complete matrix, which is executed similarly to Sputnik's approach. The non-uniform sparse distribution in W_q is more pronounced, making the payoff curve for $Q = X \times W_q$ more pronounced compared to $K = X \times W_k$ and $V = X \times W_v$, and resulting in positive performance gains at lower sparsity rates compared to cuBlas.

In summary, **CROSS** achieves better performance than cuBlas for $Q = X \times W_q$, $K = X \times W_k$, and $V = X \times W_v$ when sparsity exceeds 57.3%, 61.9%, and 66.1% respectively (red points in Figure 19). Moreover, **CROSS** lowers the sparsity level required to achieve performance gains by an average of 15.2% compared to Sputnik.

G. Preprocessing Overhead Analysis

BeTT Costs. We analyze the BeTT construction and preprocessing overhead for Llama-2-7B during compilation. Figure 20 (a) shows that BeTT construction overhead depends on the minimum profiling block shape and sampling sparse interval. Smaller blocks and intervals yield more accurate execution time models but increase construction time. We evaluated BeTT construction time and polynomial curve fitting errors for minimum profiling block shapes of 8×8 , 16×16 , and 32×32 , with sparse sampling intervals of 1%, 2%, and 4%. We opted for a 16×16 shape with 1% sparse sampling interval, balancing construction time and model accuracy. BeTT's primary function is to establish the relationship between sparsity rates and expected execution times for different block shapes. The main storage overhead consists of polynomial fitting curve parameters and block shape indices. Using 5th-order polynomial fitting, the storage overhead is approximately 80KB. Importantly, BeTT is built during compilation and only needs to be constructed once. Rebuilding is necessary only when the execution environment changes.

Preprocessing Breakdown. Figure 20 (b) illustrates the preprocessing overhead of *Sparsity Profiling & Analysis*, *Format-Variable Tensor Representation*, *Intra-batch load balance strategy*, and *Inter-batch load balance strategy* for each layer in Llama-2-7B. The overhead of Sparsity Profiling & Analysis and Format-Variable Tensor Representation (i.e., Block fusion, Block shape optimization, and sparse encoding, etc.) is the largest, accounting for 35.9% and 37.7%, respectively, while *Intra-batch load balance strategy*, and *Inter-batch load balance strategy* have smaller costs, accounting for 17.3% and 9.1%, respectively.

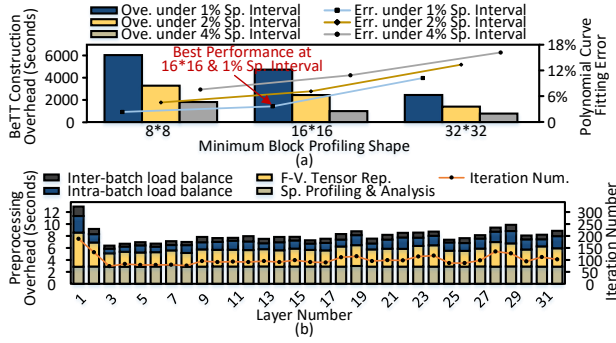


Fig. 20. Preprocessing overhead for Llama-2-7B. (a) BeTT construction overhead (Ove.) and polynomial curve fitting error (Err.) for various minimum block profiling shapes and sparsity (Sp.) intervals. (b) Preprocessing overhead for Sparsity Profiling & Analysis, Format-Variable Tensor Representation (F-V Tensor Rep.), and Intra/Inter-batch load balancing strategies.

H. Impact on Dense and Sparse Threshold.

In Section III-B, we discussed the methods for determining dense and sparse thresholds. To study their impact on performance, we analyze the performance of a sparse block with a size of 256×256 . By adjusting the dense and sparse thresholds, the range of the *Swing band* changes. This adjustment can lead to the same block being categorized differently, which affects the performance of the SpMM operation ($Q = X \times W_q$).

Figure 21 shows that the dense and sparse thresholds determined using polynomial curve fitting (67.5% for dense threshold and 83.5% for sparse threshold) align well with the optimal thresholds found experimentally (67.2% for dense threshold and 82.0% for sparse threshold). This demonstrates that the thresholds derived from curve fitting are effective for optimizing performance.

VI. RELATED WORKS

A. Existing Sparse DNN Frameworks and Compilers

In recent years, model sparsification has gained significant attention in the industry, leading to the development of various sparse model frameworks and compilation designs [17]. For instance, MT1 [4], SIPR [39], and Ironman [34] proposed specific optimizations of the kernel for specific data structures in the last century. TACO [5], [19], [20], which presents sparse format abstractions and code generation routines based on merged grids. SparTA [58] proposed a sparse model abstraction; however, its reliance on sparse libraries such as cuSPARSE can limit performance. SparseTIR [54] introduced a combinable format supporting multiple sparse formats but requires manual selection of optimal combinations, limiting its practical utility. ProSparse [43] developed a progressive sparsity regularization method that effectively increases the sparsity rate of activation layers.

B. Sparse kernel optimizations

Sparse kernel optimization is crucial for accelerating many sparse DNN frameworks and compilation designs. Various compilers have introduced their sparse kernel optimizations.

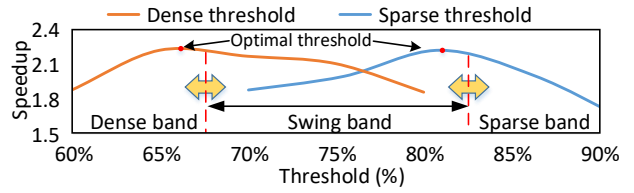


Fig. 21. The speedup of SpMM ($Q = X \times W_q$) under different thresholds (The blocks come from the W_q matrix of Llama-2-7B, and are fixedly divided into the shape of $[256, 256]$).

For instance, TVM-Sparse [1], and TACO, while others rely on other sparse libraries, such as Rammer [32]. cuSPARSE is a sparse arithmetic library proposed by NVIDIA, which can support a variety of sparse formats and sparse computation paradigms. Notably, Sputnik [12] introduces a hierarchical 1-dimensional tiling technique to enhance resource utilization. Merge-SpMM [53] focuses on latency hiding through instruction-level parallelism and load balancing. Additionally, approaches like ASpT [15], GE-SpMM [16], DA-SpMM [7], and OSKI [46] have explored different scheduling schemes for the SpMM operator.

C. Sparse format optimization

Sparse format optimization is crucial for improving the efficiency of sparse matrix computations. Several approaches have been proposed to improve the block granularity and overall performance of sparse matrices. Pichon et al. [38], Li et al. [24], Mehrabi et al. [35], and Wang et al. [49] suggest reordering the rows and columns of sparse matrices to achieve better block granularity. TSTC [41] introduces a coarse-grained tensor representation that accommodates filter-wise, channel-wise, and block-wise sparse patterns. Additionally, VCSR [18] and CSR5 [30] have developed novel sparse formats that offer significant advantages in HPC.

VII. CONCLUSION

In this work, we introduce **CROSS**, a novel system designed to optimize sparse matrix computations with a comprehensive and principled approach. **CROSS** effectively manages a wide range of sparsity patterns by integrating end-to-end optimizations that align sparse DNN computations with high-performance dense computation kernels. By incorporating profiling, optimization, and execution strategies across the entire computational stack, **CROSS** addresses both intra-batch and inter-batch load imbalances and optimizes block-level operations. This comprehensive approach enhances computational efficiency and accelerates advances in sparse matrix computations. Extensive experiments demonstrate that **CROSS** delivers substantial performance gains.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (No.62402311) and Alibaba Group through Alibaba Innovative Research Program. We sincerely thank the anonymous reviewers for their insightful suggestions. We also thank Huan Zhou for improving the figures.

REFERENCES

- [1] "Improve sparse performance on rocm." [Online]. Available: <https://github.com/apache/tvm/blob/254563a3140cf63fe77a46058688209de3aa213c/python/tvm/topi/cuda/sparse.py#L96>
- [2] "The api reference guide for cublas, the cuda basic linear algebra subroutine library." 2021. [Online]. Available: <https://docs.nvidia.com/cuda/cublas/index.html>
- [3] "cusparse: A high-performance cuda library for sparse matrix-matrix multiplication." 2021. [Online]. Available: <https://docs.nvidia.com/cuda/cusparse/index.html>
- [4] A. J. Bik and H. A. Wijshoff, "Advanced compiler optimizations for sparse computations," in *Proceedings of the 1993 ACM/IEEE conference on Supercomputing*, 1993, pp. 430–439.
- [5] S. Chou, F. Kjolstad, and S. Amarasinghe, "Format abstraction for sparse tensor algebra compilers," *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, oct 2018. [Online]. Available: <https://doi.org/10.1145/3276493>
- [6] A. Cipolletta and A. Calimera, "On the efficiency of sparse-tiled tensor graph processing for low memory usage," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 643–648.
- [7] G. Dai, G. Huang, S. Yang, Z. Yu, H. Zhang, Y. Ding, Y. Xie, H. Yang, and Y. Wang, "Heuristic adaptability to input dynamics for spmm on gpus," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 595–600. [Online]. Available: <https://doi.org/10.1145/3489517.3530508>
- [8] DataCanary, hilfalkaff, L. Jiang, M. Risdal, N. Dandekar, and tomtung, "Quora question pairs." 2017. [Online]. Available: <https://kaggle.com/competitions/quora-question-pairs>
- [9] L. Deng, G. Li, S. Han, L. Shi, and Y. Xie, "Model compression and hardware acceleration for neural networks: A comprehensive survey," *Proceedings of the IEEE*, vol. 108, no. 4, pp. 485–532, 2020.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [11] L. Floridi, "Gpt-3: Its nature, scope, limits, and consequences," dec 2020. [Online]. Available: <https://doi.org/10.1007/s11023-020-09548-1>
- [12] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse gpu kernels for deep learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '20. IEEE Press, 2020.
- [13] H. Gao, Z. Wang, and S. Ji, "Channelnets: Compact and efficient convolutional neural networks via channel-wise convolutions," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/6a4cbdaedcbda0fa8ddc7ea32073c475-Paper.pdf
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [15] C. Hong, A. Sukumaran-Rajam, I. Nisa, K. Singh, and P. Sadayappan, "Adaptive sparse tiling for sparse matrix multiplication." New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3293883.3295712>
- [16] G. Huang, G. Dai, Y. Wang, and H. Yang, "Ge-spmm: general-purpose sparse matrix-matrix multiplication on gpus for graph neural networks," ser. SC '20. IEEE Press, 2020.
- [17] S. Huang, F. Liu, T. Yang, Z. Wang, N. Yang, and L. Jiang, "Spmmplu-pro: An enhanced compiler plug-in for efficient spmm and sparsity propagation algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [18] E. Karimi, N. B. Agostini, S. Dong, and D. Kaeli, "Vcsr: An efficient gpu memory-aware sparse format," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3977–3989, 2022.
- [19] F. Kjolstad, W. Ahrens, S. Kamil, and S. Amarasinghe, "Tensor algebra compilation with workspaces," in *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, ser. CGO 2019. IEEE Press, 2019, p. 180–192.
- [20] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The tensor algebra compiler," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, oct 2017. [Online]. Available: <https://doi.org/10.1145/3133901>
- [21] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [22] K. Lagler, M. Schindelegger, J. Böhm, H. Krásná, and T. Nilsson, "Gpt2: Empirical slant delay model for radio space geodetic techniques," *Geophysical research letters*, vol. 40, no. 6, pp. 1069–1073, 2013.
- [23] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019. [Online]. Available: <https://arxiv.org/abs/1910.13461>
- [24] J. Li, B. Uçar, Ü. V. Çatalyürek, J. Sun, K. Barker, and R. Vuduc, "Efficient and effective sparse tensor reordering," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 227–237.
- [25] F. Liu *et al.*, "Sme: Reram-based sparse-multiplication-engine to squeeze-out bit sparsity of neural network," in *ICCD*, 2021.
- [26] F. Liu, Z. Wang, Y. Chen, Z. He, T. Yang, X. Liang, and L. Jiang, "Sobs-x: Squeeze-out bit sparsity for reram-crossbar-based neural network accelerator," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 1, pp. 204–217, 2022.
- [27] F. Liu, N. Yang, H. Li, Z. Wang, Z. Song, S. Pei, and L. Jiang, "Spark: Scalable and precision-aware acceleration of neural networks via efficient encoding," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 1029–1042.
- [28] F. Liu, W. Zhao, Z. He, Y. Wang, Z. Wang, C. Dai, X. Liang, and L. Jiang, "Improving neural network efficiency via post-training quantization with adaptive floating-point," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 5281–5290.
- [29] F. Liu, W. Zhao, Z. Wang, Y. Chen, X. Liang, and L. Jiang, "Era-bs: Boosting the efficiency of reram-based pim accelerator with fine-grained bit-level sparsity," *IEEE Transactions on Computers*, 2023.
- [30] W. Liu and B. Vinter, "Csr5: An efficient storage format for cross-platform sparse matrix-vector multiplication," in *Proceedings of the 29th ACM on International Conference on Supercomputing*, ser. ICS '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 339–350. [Online]. Available: <https://doi.org/10.1145/2751205.2751209>
- [31] L. Lu *et al.*, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO*, 2021.
- [32] L. Ma, Z. Xie, Z. Yang, J. Xue, Y. Miao, W. Cui, W. Hu, F. Yang, L. Zhang, and L. Zhou, "Rammer: Enabling holistic deep learning compiler optimizations with {rTasks}," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020, pp. 881–897.
- [33] H. Mao, S. Han, J. Pool, W. Li, X. Liu, Y. Wang, and W. J. Dally, "Exploring the regularity of sparse structure in convolutional neural networks," 2017. [Online]. Available: <https://arxiv.org/abs/1705.08922>
- [34] N. Mateev, K. Pingali, P. Stodghill, and V. Kotlyar, "Next-generation generic programming and its application to sparse matrix computations," in *Proceedings of the 14th international conference on Supercomputing*, 2000, pp. 88–99.
- [35] A. Mehrabi, D. Lee, N. Chatterjee, D. J. Sorin, B. C. Lee, and M. O'Connor, "Learning sparse matrix row permutations for efficient spmm on gpu architectures," in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2021, pp. 48–58.
- [36] L. Nan, D. Radev, R. Zhang, A. Rau, A. Sivaprasad, C. Hsieh, X. Tang, A. Vyas, N. Verma, P. Krishna, Y. Liu, N. Irwanto, J. Pan, F. Rahman, A. Zaidi, M. Mutuma, Y. Tarabar, A. Gupta, T. Yu, Y. C. Tan, X. V. Lin, C. Xiong, R. Socher, and N. F. Rajani, "DART: Open-domain structured data record to text generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 432–447. [Online]. Available: <https://aclanthology.org/2021.naacl-main.37>
- [37] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, "The lambada dataset," Apr. 2019. [Online]. Available: <https://doi.org/10.5281/zenodo.2630551>
- [38] G. Pichon, M. Faverge, P. Ramet, and J. Roman, "Reordering strategy for blocking optimization in sparse linear solvers," *SIAM Journal on Matrix Analysis and Applications*, vol. 38, no. 1, pp. 226–248, 2017.

- [39] W. Pugh and T. Shpeisman, "Sipr: A new framework for generating efficient code for sparse matrix computations," in *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 1998, pp. 213–229.
- [40] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "Imagenet large scale visual recognition challenge," *International journal of computer vision*, vol. 115, pp. 211–252, 2015.
- [41] T. L. Z. W. H. L. Shiyuan Huang, Fangxin Liu and L. Jiang, "Tstc: Enabling efficient training via structured sparse tensor compilation," in *Proceedings of the 29th ACM on International Conference on Supercomputing*, 2024.
- [42] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [43] C. Song, X. Han, Z. Zhang, S. Hu, X. Shi, K. Li, C. Chen, Z. Liu, G. Li, T. Yang, and M. Sun, "Prospare: Introducing and enhancing intrinsic activation sparsity within large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2402.13516>
- [44] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [45] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [46] R. Vuduc, J. W. Demmel, and K. A. Yelick, "Oski: A library of automatically tuned sparse matrix kernels," in *Journal of Physics: Conference Series*, vol. 16, no. 1. IOP Publishing, 2005, p. 521.
- [47] Wang *et al.*, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *HPCA*, 2021.
- [48] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2019, in the Proceedings of ICLR.
- [49] Y. Wang, B. Feng, G. Li, S. Li, L. Deng, Y. Xie, and Y. Ding, "{GNNAdvisor}: An adaptive and efficient runtime system for {GNN} acceleration on {GPUs}," in *15th USENIX symposium on operating systems design and implementation (OSDI 21)*, 2021, pp. 515–531.
- [50] Z. Wang, F. Liu, N. Yang, S. Huang, H. Li, and L. Jiang, "Compass: Sram-based computing-in-memory snn accelerator with adaptive spike speculation," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1090–1106.
- [51] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., vol. 29. Curran Associates, Inc., 2016. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2016/file/41bfd20a38bb1b0bec75acf0845530a7-Paper.pdf
- [52] B. T. Yamaguchi and F. Busato, "Accelerating matrix multiplication with block sparse format and nvidia tensor cores," 2021. [Online]. Available: <https://developer.nvidia.com/blog/accelerating-matrix-multiplication-with-block-sparse-format-and-nvidia-tensor-cores/>
- [53] C. Yang, A. Buluç, and J. D. Owens, "Design principles for sparse matrix multiplication on the gpu," in *Euro-Par 2018: Parallel Processing*, M. Aldinucci, L. Padovani, and M. Torquati, Eds. Cham: Springer International Publishing, 2018, pp. 672–687.
- [54] Z. Ye, R. Lai, J. Shao, T. Chen, and L. Ceze, "Sparsetir: Composible abstractions for sparse compilation in deep learning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 660–678. [Online]. Available: <https://doi.org/10.1145/3582016.3582047>
- [55] L. Yin, Y. Wu, Z. Zhang, C.-Y. Hsieh, Y. Wang, Y. Jia, G. Li, A. Jaiswal, M. Pechenizkiy, Y. Liang *et al.*, "Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity," *arXiv preprint arXiv:2310.05175*, 2023.
- [56] O. Zafrir, A. Larey, G. Boudoukh, H. Shen, and M. Wasserblat, "Prune once for all: Sparse pre-trained language models," 2021. [Online]. Available: <https://arxiv.org/abs/2111.05754>
- [57] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 863–879. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [58] N. Zheng, B. Lin, Q. Zhang, L. Ma, Y. Yang, F. Yang, Y. Wang, M. Yang, and L. Zhou, "SparTA: Deep-Learning model sparsity via Tensor-with-Sparsity-Attribute," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 213–232. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/zheng-ningxin>
- [59] M. Zhu, T. Zhang, Z. Gu, and Y. Xie, "Sparse tensor core: Algorithm and hardware co-design for vector-wise sparse neural networks on modern gpus," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 359–371. [Online]. Available: <https://doi.org/10.1145/3352460.3358269>